

西安电子科技大学

微机原理与系统设计 课程实验报告

实验名称 汇编语言编程实验

计算机科学与技术学院 2303013 班

姓名 郑墨涵 学号 23009201247

同作者 _____

实验日期 2025 年 11 月 2 日

实验地点 EII312 实验批次 计科第 3 批

成 绩

指导教师评语：

指导教师：

_____ 年 _____ 月 _____ 日

实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验所用仪器（或实验环境）
- 三、实验基本原理及步骤（或方案设计及理论计算）
- 四、实验数据记录（或仿真及软件设计）
- 五、实验结果分析及回答问题（或测试环境及测试结果）

一、实验目的

- 1. 掌握汇编语言的编程方法
- 2. 掌握 DOS 功能调用的使用方法
- 3. 掌握汇编语言程序的调试运行过程

二、实验内容

- 1. 将指定数据区的字符串数据以 ASCII 码形式显示在屏幕上，并通过 DOS 功能调用完成必要提示信息的显示。
- 2. 在屏幕上显示自己的学号姓名信息。
- 3. 循环从键盘读入字符并回显在屏幕上，然后显示出对应字符的 ASCII 码，直到输入“Q”或“q”时结束。
- 4. 自主设计输入显示信息，完成编程与调试，演示实验结果。

三、实验步骤

- 1. 运行 QTHPCI 软件，根据实验内容，参考程序流程图编写程序。
- 2. 选择“项目”菜单中的“编译”或“编译连接”对实验程序进行编译连接。
- 3. 选择“调试”菜单中的“进行调试”，进入 Debug 调试，观察调试过程中传输指令执行后各寄存器及数据区的内容。按 F9 连续运行。

四、实验过程

- 1. 完成此次实验，需要对汇编语言中的系统功能调用有一些了解，可能使用到的系统功能调用如下所示。注意使用如下系统功能调用时，需要与 INT 21H 一同使用。INT 是 interupt 中断的缩写，是 DOS 的中断调用命令。

本次实验的任务二，显示学号姓名信息，就需要用到 int 21H 中断的 09H 号功能。

将 DX 寄存器设置为待显示的字符串偏移地址，将 AH 寄存器的内容设置为 09 调用 int 21H 中断，就可以把待显示字符串显示到屏幕上。

AH 值	功 能	调用参数	结 果
1	键盘输入并回显		AL=输出字符
2	显示单个字符(带 Ctrl+Break 检查)	DL=输出字符	光标在字符后面
6	显示单个字符(无 Ctrl+Break 检查)	DL=输出字符	光标在字符后面
8	从键盘上读一个字符		AL=字符的ASCII码
9	显示字符串	DS:DX=串地址，‘\$’为结束字符	光标跟在串后面
4CH	返回DOS系统		AL=返回码

图 1. 部分系统功能调用参考表

- 2. 在计算机中，所有的数据均以二进制 01 存储，其中字符则存放其对应的 ASCII 码值，读取数据时，寄存器中存放的值均为 ASCII 码值。实验要求输出其 ASCII 码，而被输出的 ASCII 码又是以 ASCII 码表示的。简而言之，需要做两次关于字符与 ASCII 码的映射。
- 3. 当 X 或 Y 值为 0~9H 时，需要加上调整值 30H；当 X 或 Y 值为 A~FH 时，需要加上调整值 37H。 据此，则可将其进行转换为 ASCII 码值。

4. 代码

DATA SEGMENT	LEA DX, SEPARATOR	
WELCOME_MSG DB '==	INT 21H	JMP MAIN_LOOP_START
Character and ASCII Code Display		
Program ==', 0DH, 0AH, '\$'	RET	EXIT_MAIN_LOOP:
PROMPT_MSG DB 'Please	DISPLAY_WELCOME ENDP	MOV AH, 09H
enter a character (Q/q to quit): \$'		LEA DX, NEWLINE
CHAR_MSG DB 'The character	DISPLAY_STUDENT_INFO PROC	INT 21H
is: \$'	MOV AH, 09H	LEA DX, SEPARATOR
ASCII_MSG DB ' ASCII code	LEA DX, STUDENT_INFO	INT 21H
(hex): \$'	INT 21H	
NEWLINE DB 0DH, 0AH, '\$'		LEA DX, EXIT_MSG
SEPARATOR DB	LEA DX, SEPARATOR	INT 21H
'-----',	INT 21H	
0DH, 0AH, '\$'		RET
STUDENT_INFO DB 'Student	RET	MAIN_LOOP ENDP
ID: 23009201247 Name:	DISPLAY_STUDENT_INFO ENDP	
ZhengMohan', 0DH, 0AH, '\$'		DISPLAY_CHAR_INFO PROC
	MAIN_LOOP PROC	MOV AH, 09H
CHAR_BUFFER DB ?	MAIN_LOOP_START:	LEA DX, CHAR_MSG
ASCII_BUFFER DB 3 DUP('\$')		INT 21H
DATA ENDS	MOV AH, 09H	
	LEA DX, PROMPT_MSG	MOV DL, CHAR_BUFFER
CODE SEGMENT	INT 21H	MOV AH, 02H
ASSUME CS:CODE, DS:DATA		INT 21H
	MOV AH, 01H	
START:	INT 21H	MOV AH, 09H
MOV AX, DATA		LEA DX, ASCII_MSG
MOV DS, AX	MOV CHAR_BUFFER, AL	INT 21H
CALL DISPLAY_WELCOME		
	MOV AH, 09H	MOV AL, CHAR_BUFFER
CALL	LEA DX, NEWLINE	CALL DISPLAY_HEX
DISPLAY_STUDENT_INFO	INT 21H	
		MOV AH, 09H
CALL MAIN_LOOP	CMP CHAR_BUFFER, 'Q'	LEA DX, NEWLINE
	JE EXIT_MAIN_LOOP	INT 21H
MOV AH, 4CH	CMP CHAR_BUFFER, 'q'	
INT 21H	JE EXIT_MAIN_LOOP	RET
		DISPLAY_CHAR_INFO ENDP
DISPLAY_WELCOME PROC	CALL DISPLAY_CHAR_INFO	
MOV AH, 09H		DISPLAY_HEX PROC
LEA DX, WELCOME_MSG	MOV AH, 09H	MOV BL, AL
INT 21H	LEA DX, SEPARATOR	
	INT 21H	MOV AL, BL

SHR AL, 4		ADD AL, '0'
CALL HEX_DIGIT_TO_ASCII	RET	
MOV DL, AL	DISPLAY_HEX ENDP	CONVERSION_DONE:
MOV AH, 02H		RET
INT 21H	HEX_DIGIT_TO_ASCII PROC	HEX_DIGIT_TO_ASCII ENDP
	CMP AL, 10	
MOV AL, BL	JB IS_DIGIT	EXIT_MSG DB 'Program ended.
AND AL, 0FH		Thank you for using!', 0DH, 0AH,
CALL HEX_DIGIT_TO_ASCII	ADD AL, 'A' - 10	'\$'
MOV DL, AL	JMP CONVERSION_DONE	
MOV AH, 02H		CODE ENDS
INT 21H	IS_DIGIT:	END START

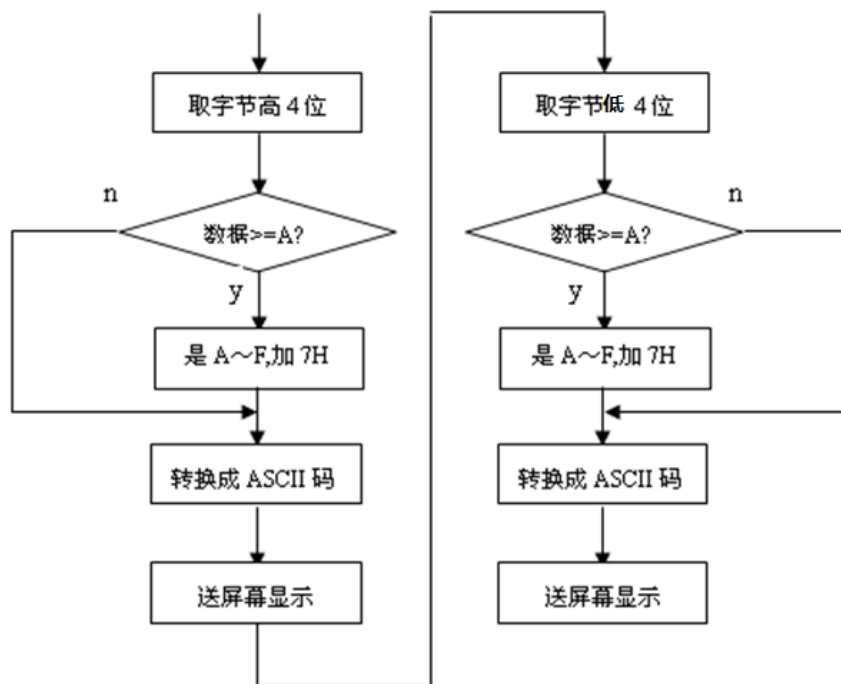


图 3. 字符转换为 ASCII 码流程图

西安电子科技大学

微机原理与系统设计 课程实验报告

实验名称 数码转换实验

计算机科学与技术学院 2303013 班

姓名 郑墨涵 学号 23009201247

同作者 _____

实验日期 2025 年 11 月 9 日

实验地点 EII312 实验批次 计科第 3 批

成 绩

指导教师评语：

指导教师：

_____年____月____日

实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验所用仪器（或实验环境）
- 三、实验基本原理及步骤（或方案设计及理论计算）
- 四、实验数据记录（或仿真及软件设计）
- 五、实验结果分析及回答问题（或测试环境及测试结果）

一、实验目的

5. 掌握不同进制数及编码相互转换的程序设计方法。
6. 掌握运算类指令编程及调试方法。
7. 掌握循环程序的设计方法。

二、实验内容

8. 重复从键盘输入不超过 5 位的十进制数，按回车键结束输入；
2. 将该十进制数转换成二进制数；结果以 2 进制数的形式显示在屏幕上；
3. 如果输入非数字字符，则报告出错信息，重新输入；
4. 直到输入“Q”或‘q’时程序运行结束。
5. 键盘输入一字符串，以空格结束，统计其中数字字符的个数，在屏幕显示。

三、实验原理

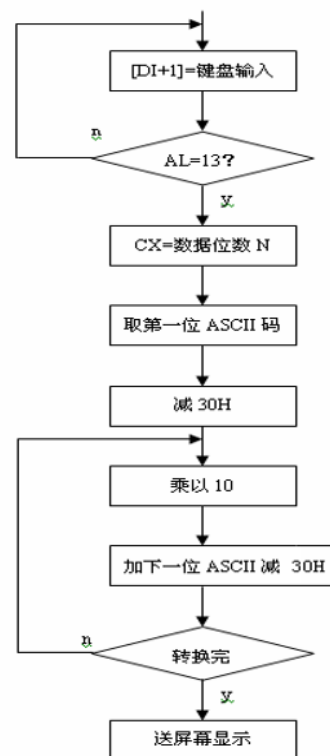
十进制数可以表示为： $D_n * 10^n + D_{n-1} * 10^{n-1} + \dots + D_0 * 10^0 = \sum D_i * 10^i$

其中 D_i 表十进制数 1、2、3、...、9、0。

上式可以转换为： $\sum D_i * 10^i = ((D_n * 10 + D_{n-1}) * 10 + D_{n-2}) * 10 + \dots + D_1) * 10 + D_0$

由上式可归纳出十进制数转换为二进制数的方法：从十进制数的最高位 D_n 开始做乘 10 加次位的操作，依此类推，则可求出二进制数结果。转换过程可参考图 2.1 十进制 ASCII 码转换为二进制数流程图

十进制ASCII码转换为二进制数流程图



四、实验代码

DATA SEGMENT	MOV DS, AX	JMP MAIN_LOOP
INPUT_MSG DB 'Enter a decimal number (up to 10 digits, Q to quit): \$'	MAIN_LOOP:	INPUT_ERROR:
BINARY_MSG DB '32-bit Binary: \$'	MOV AH, 09H	MOV AH, 09H
ERROR_MSG DB 'Error: Invalid input! Please enter digits only.', 0DH, 0AH, '\$'	LEA DX, INPUT_MSG	LEA DX, ERROR_MSG
LENGTH_ERROR_MSG DB 'Error: Number exceeds 10 digits or 32-bit range!', 0DH, 0AH, '\$'	INT 21H	INT 21H
COUNT_MSG DB 'Number of digits in string: \$'	MOV AH, 0AH	JMP MAIN_LOOP
STRING_PROMPT DB 'Enter a string (space to finish): \$'	LEA DX, EXIT_PROGRAM:	CALL
NEWLINE DB 0DH, 0AH, '\$'	DECIMAL_BUFFER	COUNT_DIGITS_IN_STRING
PRESS_ANY_KEY DB 'Press any key to exit...\$'	INT 21H	
	MOV AH, 09H	MOV AH, 4CH
	LEA DX, NEWLINE	INT 21H
	INT 21H	
	MOV SI, OFFSET	VALIDATE_INPUT PROC
	DECIMAL_BUFFER + 2	MOV SI, OFFSET
	MOV AL, [SI]	DECIMAL_BUFFER + 2
	CMP AL, 'Q'	MOV CL,
	JE EXIT_PROGRAM	DECIMAL_BUFFER + 1
	CMP AL, 'q'	CMP CL, 0
	JE EXIT_PROGRAM	JE VALIDATE_FAIL
	MOV AL,	VALIDATE_LOOP:
	DECIMAL_BUFFER + 1 ;	CMP CL, 0
	实际输入的字符数	JE VALIDATE_SUCCESS
	CMP AL, 10	
	JA LENGTH_ERROR	MOV AL, [SI]
	CALL VALIDATE_INPUT	CMP AL, '0'
	JC INPUT_ERROR	JB VALIDATE_FAIL
	CALL	CMP AL, '9'
	CONVERT_TO_BINARY	JA VALIDATE_FAIL
	JMP MAIN_LOOP	INC SI
		DEC CL
		JMP VALIDATE_LOOP
CODE SEGMENT	LENGTH_ERROR:	VALIDATE_SUCCESS:
ASSUME CS:CODE,	MOV AH, 09H	CLC
DS:DATA	LEA DX,	RET
START:	LENGTH_ERROR_MSG	
MOV AX, DATA	INT 21H	

VALIDATE_FAIL:	MOV SI, OFFSET	MOV AX, NUMBER_LOW
STC	DECIMAL_BUFFER + 2	MOV DX, NUMBER_HIGH
RET	MOV CL,	
VALIDATE_INPUT ENDP	DECIMAL_BUFFER + 1 ;	; 乘以 2
	字符数	SHL AX, 1
CONVERT_TO_BINARY PROC		RCL DX, 1
CALL	MOV NUMBER_LOW, 0	
ASCII_TO_32BIT_NUMBER	MOV NUMBER_HIGH, 0	MOV CX, DX
		MOV BX, AX
CMP NUMBER_HIGH, 0	CONVERT_ASCII_LOOP:	
JNE VALUE_OK	CMP CL, 0	SHL AX, 1
	JE CONVERT_DONE	RCL DX, 1
CMP NUMBER_LOW, 65535		SHL AX, 1
JBE VALUE_OK	MOV AL, [SI]	RCL DX, 1
	SUB AL, '0'	SHL AX, 1
MOV AH, 09H	MOV AH, 0	RCL DX, 1
LEA DX,	MOV DX, AX	
LENGTH_ERROR_MSG		ADD AX, BX
INT 21H	;	ADC DX, CX
RET	NUMBER_HIGH:NUMBER_LOW =	
	NUMBER_HIGH:NUMBER_LOW *	MOV NUMBER_LOW, AX
VALUE_OK:	10	MOV NUMBER_HIGH, DX
MOV AH, 09H	CALL	
LEA DX, BINARY_MSG	MULTIPLY_32BIT_BY_10	POP CX
INT 21H		POP DX
	ADD NUMBER_LOW, DX	POP AX
MOV AX, NUMBER_HIGH	ADC NUMBER_HIGH, 0	RET
MOV DX, NUMBER_LOW		MULTIPLY_32BIT_BY_10
CALL	INC SI	ENDP
CONVERT_32BIT_TO_BINARY	DEC CL	
	JMP	CONVERT_32BIT_TO_BINARY
MOV AH, 09H	CONVERT_ASCII_LOOP	PROC
LEA DX, BINARY_BUFFER		MOV DI, OFFSET
INT 21H	CONVERT_DONE:	BINARY_BUFFER
	RET	MOV CX, 32
MOV AH, 09H	ASCII_TO_32BIT_NUMBER	CONVERT_LOOP:
LEA DX, NEWLINE	ENDP	SHL AX, 1
INT 21H		RCL DX, 1
	MULTIPLY_32BIT_BY_10	JC SET_ONE
RET	PROC	
CONVERT_TO_BINARY ENDP	PUSH AX	SET_ZERO:
	PUSH DX	MOV BYTE PTR [DI], '0'
ASCII_TO_32BIT_NUMBER	PUSH CX	JMP NEXT_BIT
PROC		

SET_ONE:	JA NOT_DIGIT	DISPLAY_NUMBER PROC
MOV BYTE PTR [DI], '1'		MOV BX, 10
	INC CX	MOV CX, 0
NEXT_BIT:	JMP READ_STRING	
INC DI		
LOOP CONVERT_LOOP	NOT_DIGIT:	CMP AX, 0
	JMP READ_STRING	JNE PUSH_DIGITS
MOV BYTE PTR [DI], '\$'		
RET	DISPLAY_COUNT:	MOV DL, '0'
CONVERT_32BIT_TO_BINARY	MOV AH, 09H	MOV AH, 02H
ENDP	LEA DX, NEWLINE	INT 21H
	INT 21H	RET
COUNT_DIGITS_IN_STRING		
PROC	LEA DX, COUNT_MSG	PUSH_DIGITS:
MOV AH, 09H	INT 21H	MOV DX, 0
LEA DX, NEWLINE		DIV BX
INT 21H	MOV AX, CX	PUSH DX
	CALL DISPLAY_NUMBER	INC CX
LEA DX, STRING_PROMPT		CMP AX, 0
INT 21H	MOV AH, 09H	JNE PUSH_DIGITS
	LEA DX, NEWLINE	
MOV CX, 0	INT 21H	POP_DIGITS:
		POP DX
READ_STRING:	MOV AH, 09H	ADD DL, '0'
MOV AH, 01H	LEA DX, NEWLINE	MOV AH, 02H
INT 21H	INT 21H	INT 21H
		LOOP POP_DIGITS
CMP AL, ' '	LEA DX, PRESS_ANY_KEY	
JE DISPLAY_COUNT	INT 21H	RET
		DISPLAY_NUMBER ENDP
CMP AL, 0DH	MOV AH, 07H	
JE DISPLAY_COUNT	INT 21H	CODE ENDS
		END START
CMP AL, '0'	RET	
JB NOT_DIGIT	COUNT_DIGITS_IN_STRING	
CMP AL, '9'	ENDP	

西安电子科技大学

微机原理与系统设计 课程实验报告

实验名称 基本 IO 拓展实验

计算机科学与技术学院 2303013 班

姓名 郑墨涵 学号 23009201247

同作者 _____

实验日期 2025 年 11 月 16 日

实验地点 EII312 实验批次 计科第 3 批

成 绩

指导教师评语：

指导教师：

_____ 年 _____ 月 _____ 日

实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验所用仪器（或实验环境）
- 三、实验基本原理及步骤（或方案设计及理论计算）
- 四、实验数据记录（或仿真及软件设计）
- 五、实验结果分析及回答问题（或测试环境及测试结果）

一、实验目的

9. 了解 TTL 芯片扩展简单 I/O 口的方法。
10. 掌握数据输入输出程序编制的方法。

二、实验内容说明

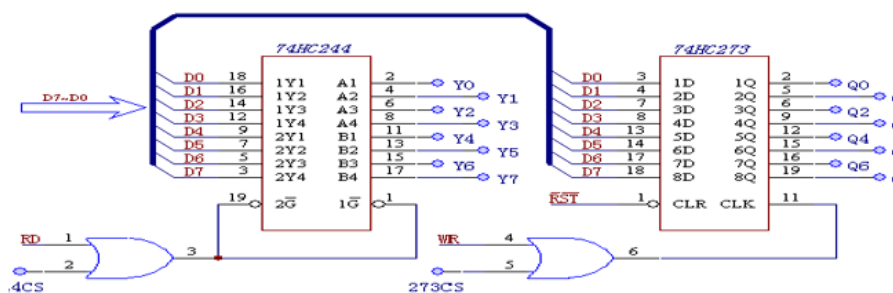
本实验要求用 74LS244 作为输入口，读取开关状态，并将此状态通过 74LS273 连到发光二极管显示。具体实验内容如下：

11. 开关 Y_i 为低电平时对应的发光二极管亮， Y_i 为高电平时对应的发光二极管灭。
12. 当开关 Y_i 全为高电平时，发光二极管 Q_i 从左至右轮流点亮。
13. 当开关 Y_i 全为低电平时，发光二极管 Q_i 从右至左轮流点亮。
14. 主设计控制及显示模式，完成编程调试，演示实验结果。

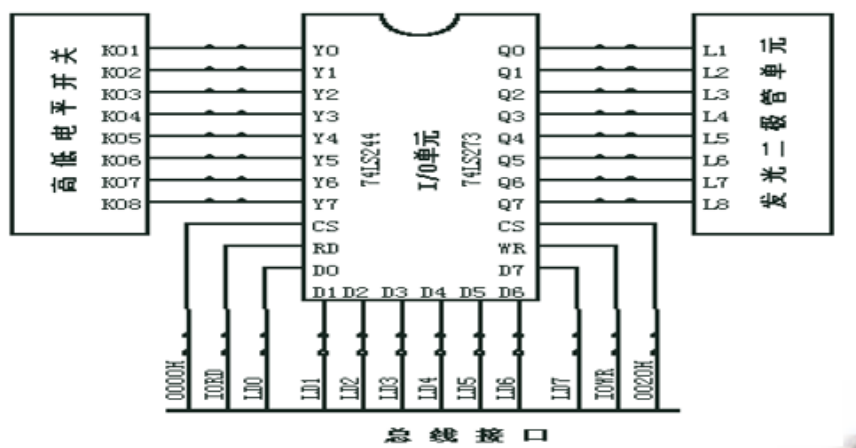
三、实验原理

74LS244 是一种三态输出的 8 总线缓冲驱动器，无锁存功能，当 G 为低电平， A_i 信号传送到 Y_i ，当为高电平时， Y_i 处于禁止高阻状态；

74LS273 是一种带清除功能的 8D 触发器， $1D \sim 8D$ 为数据输入端， $1Q \sim 8Q$ 为数据输出端，正脉冲触发，低电平清除，常用作 8 位地址锁存器。



74LS244与74LS273扩展I/O口原理图



四、实验步骤

15. 按照实验连线图连接：

- 244 的 CS 接到 ISA 总线接口模块的 0000H，Y7—Y0——开关 K1—K8。
- 273 的 CS 接到 ISA 总线接口模块的 0020H，Q7—Q0——发光二极管 L1—L8。
- 该模块的 WR、RD 分别连到 ISA 总线接口模块的 IOWR、IORD。
- 该模块的数据(AD0~AD7)连到 ISA 总线接口模块的数据(LD0~LD7)。

16. 编写实验程序，编译链接，运行程序

17. 拨动开关，观察发光二极管的变化。

五、实验结果

编译链接项目后，改变开关 Yi 可观察到

开关 Yi 为低电平时对应的发光二极管亮，Yi 为高电平时对应的发光二极管灭。

当开关 Yi 全为高电平时，发光二极管 Qi 从左至右轮流点亮。

当开关 Yi 全为低电平时，发光二极管 Qi 从右至左轮流点亮。

实验代码

```
I0244 EQU 0230H
I0273 EQU 0230H
STACK1 SEGMENT STACK
    DW 100 DUP(?)
STACK1 ENDS
```

```
DATA SEGMENT WORD PUBLIC 'DATA'
DATA ENDS
```

```
CODE SEGMENT
ASSUME CS:CODE, DS:DATA, SS:STACK1
START PROC NEAR
    MOV AX, DATA
    MOV DS, AX
```

```
INPUT:
    MOV DX, I0244
    IN AL, DX
    CMP AL, 0FFH ; 左到右
    JZ X1
    CMP AL, 000H ; 右到左
    JZ X2
    MOV DX, I0273
    OUT DX, AL
    JMP INPUT
```

```
X1:
    MOV AL, 0FEH
    MOV DX, I0273
```

```
LR:
    CALL DELAY
    OUT DX, AL
    ROL AL, 1
    CMP AL, 0FEH
    JNE LR
    JMP INPUT
```

```
X2:
    MOV AL, 07FH
    MOV DX, I0273
```

```
RL:
    CALL DELAY
    OUT DX, AL
    ROR AL, 1
    CMP AL, 07FH
    JNE RL
    JMP INPUT
```

```
DELAY PROC NEAR
```

```
Delay1:
    XOR CX, CX
    LOOP $
    RET
```

```
Delay ENDP
```

```
START ENDP
CODE ENDS
END START
```

西安电子科技大学

微机原理与系统设计 课程实验报告

实验名称 可编程并行接口 8255 实验

计算机科学与技术学院 2303013 班

姓名 郑墨涵 学号 23009201247

同作者

实验日期 2025 年 11 月 23 日

实验地点 EII312 实验批次 计科第 3 批

成 绩

指导教师评语：

指导教师：

年 月 日

实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验所用仪器（或实验环境）
- 三、实验基本原理及步骤（或方案设计及理论计算）
- 四、实验数据记录（或仿真及软件设计）
- 五、实验结果分析及回答问题（或测试环境及测试结果）

一、实验目的

了解可编程并行接口 8255 的内部结构

掌握工作方式、初始化编程及应用。

二、实验内容

流水灯实验：利用 8255 的 A 口、B 口循环点亮发光二极管。

交通灯实验：利用 8255 的 A 口模拟交通信号灯。

I/O 输入输出实验：利用 8255 的 A 口读取开关状态，8255 的 B 口把状态送发光二极管显示。

在完成(1)基础上，增加通过读取开关控制流水灯的循环方向和循环方式。

在完成(2)基础上，增加通过读取开关控制交通红绿灯的亮灭时间。

三、实验原理

1. 8255A 的内部结构

(1) 数据总线缓冲器：这是一个双向三态的 8 位数据缓冲器，它是 8255A 与微机系统数据总线的接口。输入输出的数据、CPU 输出的控制字以及 CPU 输入的状态信息都是通过这个缓冲器传送的。

(2) 三个端口 A，B 和 C：A 端口包含一个 8 位数据输出锁存器和缓冲器，一个 8 位数据输入锁存器。B 端口包含一个 8 位数据输入/输出锁存器和缓冲器，一个 8 位数据输入缓冲器。C 端口包含一个 8 位数据输出锁存器及缓冲器，一个 8 位数据输入缓冲器（输入没有锁存器）。

(3) A 组和 B 组控制电路：这是两组根据 CPU 输出的控制字控制 8255 工作方式的电路，它们对于 CPU 而言，共用一个端口地址相同的控制字寄存器，接收 CPU 输出的一字节方式控制字或对 C 口按位复位字命令。方式控制字的高 5 位决定 A 组工作方式，低 3 位决定 B 组的工作方式。对 C 口按位复位命令字可对 C 口的每一位实现置位或复位。A 组控制电路控制 A 口和 C 口上半部，B 组控制电路控制 B 口和 C 口下半部。

(4) 读写控制逻辑：用来控制把 CPU 输出的控制字或数据送至相应端口，也由它来控制把状态信息或输入数据通过相应的端口送到 CPU。

2. 8255A 的工作方式

方式 0—基本输入输出方式；方式 1—选通输入输出方式；方式 2—双向选通输入输出方式。

3. 8255A 的控制字

表 4-4-1 8255A 方式控制字

1	D6	D5	D4	D3	D2	D1	D0
特征	A 组方式		A 口	C 口高 4 位	B 组方式	B 口	C 口低 4 位
位	00=方式 0 01=方式 1		0=输出	0=输出	0=方式 0	0=输出	0=输出
	1X=方式 2		1=输入	1=输入	1=方式 1	1=输入	1=输入

表 4-4-2 按位置位/复位控制字

0	D6	D5	D4	D3	D2	D1	D0
特征位	不用			位选择			0=复位
				000=C 口 0 位……111=C 口 7 位			1=置位

4. 8255A 的状态字

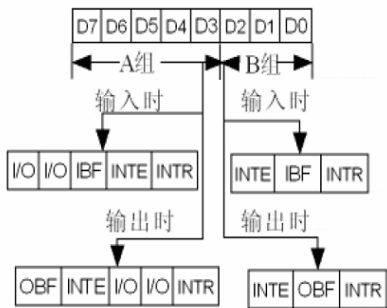


图 4-4-3 8255 方式 1 的状态字

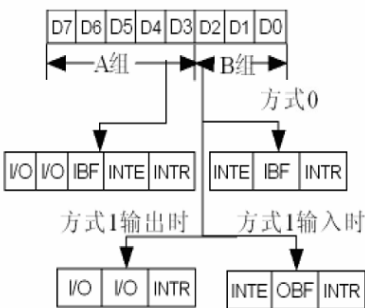
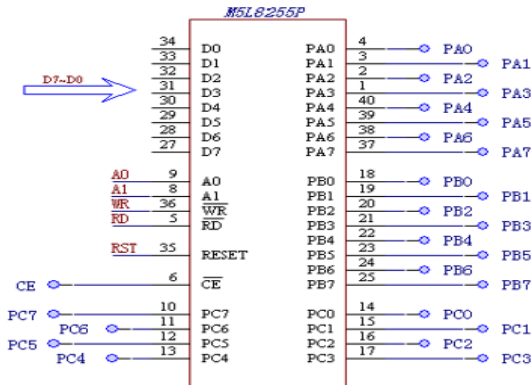


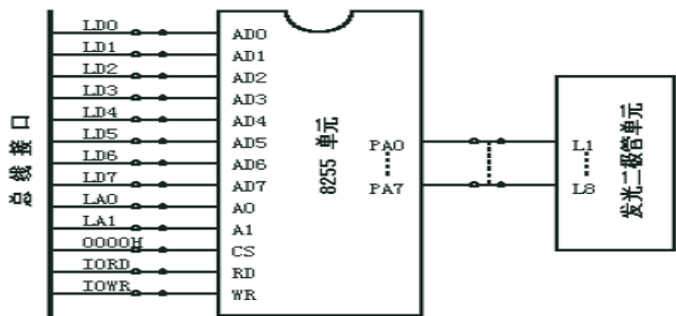
图 4-4-4 8255 方式 2 的状态字

8255 是一个通用可编程并行接口电路。它具有 A、B、C 三个 8 位并行口。其中 C 口也可用作 A、B 口的联络信号及中断申请信号。通过编程，它可以被设置为基本输入输出、选通输入输出以及双向传送方式。对于 C 口还具有按位置 0、1 的功能。

可编程并行接口8255芯片接口电路



四、实验步骤



- 模块的 WR、RD 分别连到 ISA 总线接口模块的 IOWR、IORD。
- 模块的数据 (AD0~AD7)、地址线 (A0~A7) 分别连到 ISA 总线接口模块的数据 (LD0~LD7)、地址线 (LA0~LA7)。
- 8255 模块选通线 CE 连到 ISA 总线接口模块的 0000H。
- 8255 的 PA0~PA7 连到发光二极管的 L0~L7; 8255 的 PB0~PB7 连到发光二极管的 L8~L15。
- 编写 8255 驱动程序
- 运行程序，观察发光二极管。

五、实验结果

全速运行程序后，可观察到发光二极管被循环点亮

实验代码

```
COM_ADD EQU 0273H
PA_ADD EQU 0270H
PB_ADD EQU 0271H
PC_ADD EQU 0272H
_STACK SEGMENT STACK
    DW 100 DUP(?)
_STACK ENDS
_DATA SEGMENT WORD PUBLIC 'DATA'
_DATA ENDS
CODE SEGMENT
START PROC NEAR
    ASSUME CS:CODE, DS:_DATA, SS:_STACK
    MOV AX,_DATA
    MOV DS,AX
    NOP
    MOV DX,COM_ADD
    MOV AL,82H
    OUT DX,AL
INPUT:
    MOV     AX, 0FFFFH
    MOV     DX, PA_ADD
    OUT     DX, AX
    MOV     DX, PC_ADD
    OUT     DX, AX
    ; 输入操作
    MOV     DX, PB_ADD
    IN      al, DX

    mov     ah, 0
    CMP al, 0D0H
    JZ mid
    cmp al, 0FH
    JZ lar1
    cmp al, 0F0H
    JZ ral1
    CMP al, 0FFH ;全 1,
    JZ low1
    CMP al, 0 ;全 0
    JZ high1
    MOV     DX, PA_ADD
    OUT DX, al
    JMP INPUT
mid:
    shl al,4
    shr al,4
    shr ah,4
    add al,ah
    mov ah,0
    mov dx, PA_ADD ;00001101
    out dx,ax
    jmp INPUT
ral1:
    MOV     al, 7FH
    MOV     DX, PA_ADD
```

ral2:	MOV	AX, 0FFFFH		
	ROR al, 1	OUT	DX, AX	
	OUT DX, al			
	CALL	Delay	lar3:	
	CMPal, 7FH	MOV	al, 7FH	
	JNE low2	MOV	DX, PC_ADD	
	MOV	AX, 0FFFFH	lar4:	
	OUT	DX, AX	ROR al, 1	
ral3:		OUT DX, al		
	MOV	al, 7FH	CALL	Delay
	MOV	DX, PC_ADD	CMPal, 7FH	
ral4:		JNE low4	JMP INPUT	
	ROL al, 1			
	OUT DX, al			
	CALL	Delay	low1:	
	CMPal, 7FH	MOV	al, 7FH	
	JNE low4	MOV	DX, PA_ADD	
	JMP INPUT	low2:		
		ROL al, 1		
lar1:		OUT DX, al		
	MOV	al, 7FH	CALL	Delay
	MOV	DX, PA_ADD	CMPal, 7FH	
lar2:		JNE low2		
	ROL al, 1	MOV	AX, 0FFFFH	
	OUT DX, al	OUT	DX, AX	
	CALL	Delay	low3:	
	CMPal, 7FH	MOV	al, 7FH	
	JNE low2	MOV	DX, PC_ADD	

low4:	MOV	al, 0FEH	
	ROL	al, 1	MOV
	OUT	DX, al	DX, PA_ADD
	CALL	Delay	high4:
	CMP	al, 7FH	ROR
	JNE	low4	al, 1
	JMP	INPUT	OUT
			DX, al
			CALL
			Delay
			CMP
			al, 0FEH
			JNE
			high4
high1:			JMP
			INPUT
	MOV	al, 0FEH	
	MOV	DX, PC_ADD	Delay
			PROC NEAR
high2:			Delay1:
	ROR	al, 1	XOR
	OUT	DX, al	CX, CX
	CALL	Delay	LOOP
	CMP	al, 0FEH	\$
	JNE	high2	RET
	MOV	AX, 0FFFFH	Delay
	OUT	DX, AX	ENDP
			START ENDP
			CODE ENDS
high3:			END START

实验心得

这次的微机原理课程给我提供了动手实验的机会，我详细的了解了 74LS244、74LS273、可编程并行接口 8255 的电路设计，驱动程序的编写方式，并通过自己的亲自实践，使用编写的代码完成了对 LED 发光二极管的控制。我对计算机组成原理的相关知识有了更深的印象和认识。这门课对于使我们了解现代计算机的各个组成部分及其工作原理具有重要作用，对于我们后续课程的学习无疑也具有积极的意义。我将继续努力对计算机原理方面进行深入的研究，了解更多计算机方面的知识，为以后打下坚实的基础。